Product CRUD App — Express, Mongoose & EJS

A server-rendered CRUD application: routes, controllers, and request flow explained from the source code.

1. Overview

This app is a product catalogue with full create, read, update, and delete. Express handles routing, Mongoose talks to MongoDB, and EJS templates turn data into HTML — the Express + EJS server-side rendering pattern. Every action ends in either res.render() (show a page) or res.redirect() (send the browser back to the product list).

Architecture

```
Browser
   |  GET /  or  POST /add-product ...
   v
Entry file (server.js)        maps each URL to a controller
   |
   v
productController.js          one async function per action
   |
   v
Product model (Mongoose)  -->  MongoDB
   |
   v
EJS view (res.render)         builds the HTML page
   |
   v
Browser
```

Project Structure

```
project/
├── server.js                 entry file (routes and setup)
├── .env                      MONGO_URI, DB_NAME, PORT
├── config/
│   └── db.js                 connectDB()  (imported, not shown)
├── controllers/
│   └── productController.js  the six handlers
├── models/
│   └── productModel.js       Product schema (imported, not shown)
└── views/
    ├── products.ejs          list of all products
    ├── product.ejs           one product
    ├── products-form.ejs     add / edit form
    └── error.ejs             error message page
```

The entry file name is not given in the source; server.js is used here. config/db.js, the Product model, and the EJS views are referenced by the code but were not provided, so their contents are not described. The view file names are taken from the res.render() calls.

2. Server Setup

Imports & Environment

The code uses ES module imports, so package.json needs "type": "module" — which also allows the top-level await used below. dotenv.config() loads the .env file into process.env, and connectDB() receives the connection string and database name. It is awaited before the app is created, so the server never starts without a database.

Syntax

```
import <name> from "<module>";
dotenv.config();
await connectDB(<uri>, <dbName>);
```

Example

```
import express from "express";
import {
  createProduct,
  readProducts,
  readProductById,
  updateProductForm,
  updateProduct,
  deleteProduct,
} from "./controllers/productController.js";
import dotenv from "dotenv";
import connectDB from "./config/db.js";

dotenv.config();
await connectDB(process.env.MONGO_URI, process.env.DB_NAME);
```

Middleware & View Engine

express.json() parses JSON request bodies. express.urlencoded({ extended: true }) parses HTML form submissions — the default encoding for a <form> — and fills req.body. app.set("view engine", "ejs") tells Express to look for views/<name>.ejs when res.render("<name>") is called.

Syntax

```
app.use(<middleware>);
app.set("view engine", "<engine>");
```

Example

```
const app = express();
app.use(express.json());

app.set("view engine", "ejs");
app.use(express.urlencoded({ extended: true }));
```

Starting the Server

The port comes from the PORT environment variable, falling back to 3000 when it is not set. app.listen() starts accepting requests and runs its callback once the server is ready.

Syntax

```
app.listen(<port>, <callback>);
```

Example

```
const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
  console.log(`Server is running at http://localhost:${PORT}`);
});
```

3. Routes

Seven routes cover the whole app. Five are GET routes — four show a page and one deletes a product, then redirects — and the two form submissions are POST routes.

Method	Path	Handler	Purpose
GET	/	readProducts	List all products
GET	/products-form	inline handler	Show an empty form to add a product
GET	/product/:id	readProductById	Show one product
POST	/add-product	createProduct	Save a new product
GET	/products-form/:id	updateProductForm	Show the form filled with a product to edit
POST	/add-product/:id	updateProduct	Save the edited product
GET	/delete-product/:id	deleteProduct	Delete a product

Reusing One Form for Add and Edit

The "/products-form" route has no controller — it renders the form straight from an inline handler, passing product: null. The edit route renders the same template with a real product. Inside products-form.ejs, null means "add mode" and an object means "edit mode" (fields pre-filled).

Syntax

```
res.render("<view>", { <data> });
```

Example

```
// Route to render the product form for adding a new product
app.get("/products-form", (req, res) => {
  res.render("products-form", { product: null });
});
```

Wiring Routes to Controllers

Each remaining route passes a controller function straight to app.get() or app.post(). Express calls it with (req, res) when the URL and method match, and :id in the path becomes req.params.id.

Syntax

```
app.get(<path>, <controller>);
app.post(<path>, <controller>);
```

Example

```
// Route to display all products
app.get("/", readProducts);
// Route to display a single product
app.get("/product/:id", readProductById);
// Route to handle adding a new product
app.post("/add-product", createProduct);
// Route to render the product form for editing
app.get("/products-form/:id", updateProductForm);
// Route to handle updating an existing product
app.post("/add-product/:id", updateProduct);
// Route to handle deleting a product
app.get("/delete-product/:id", deleteProduct);
```

Why POST and GET only? HTML forms can only submit GET or POST, so updating uses POST and deleting is a plain link (GET) instead of PUT and DELETE. A JSON API, as in the MongoDB with Express reference guide, would use PUT/PATCH/DELETE.

4. Controllers

Controller Setup & Exports

productController.js imports the Product model and defines one async arrow function per action. Each function is exported by name and imported by the entry file. Every function wraps its work in try/catch and, on failure, renders the error view with the error message.

Syntax

```
import <Model> from "<path>";
const <name> = async (req, res) => { ... };
export { <name1>, <name2>, ... };
```

Example

```
import Product from "../models/productModel.js";

// ... six controller functions ...
```

```
export {
  createProduct,
  readProducts,
  readProductById,
  updateProductForm,
  updateProduct,
  deleteProduct,
};
```

Create — createProduct()

Reads the form fields from req.body, builds a new Product document, and saves it. Destructuring only name, description, category, and price means any extra fields a user adds to the request are ignored. On success the browser is redirected to the list.

Syntax

```
new Product(<fields>).save()
```

Example

```
const createProduct = async (req, res) => {
  try {
    const { name, description, category, price } = req.body;
    const product = new Product({
      name,
      description,
      category,
      price,
    });
    await product.save();
    res.redirect("/");
  } catch (error) {
    res.render("error", { error: error.message });
  }
};
```

Read — readProducts()

Product.find({}) with an empty filter returns every product as an array, which is passed to the products view as products.

Syntax

```
Product.find(<filter>)
```

Example

```
const readProducts = async (req, res) => {
  try {
    const products = await Product.find({});
    res.render("products", { products });
```

```
  } catch (error) {
    res.render("error", { error: error.message });
  }
};
```

Read — readProductById()

Takes the id from the URL (req.params.id) and looks the product up with findById(). The single product is passed to the product view.

Syntax

```
Product.findById(<id>)
```

Example

```
const readProductById = async (req, res) => {
  try {
    const { id } = req.params;
    const product = await Product.findById(id);
    res.render("product", { product });
  } catch (error) {
    res.render("error", { error: error.message });
  }
};
```

Update — updateProductForm()

First half of an edit: loads the existing product and renders the shared form with it, so the fields come pre-filled.

Syntax

```
Product.findById(<id>)
```

Example

```
const updateProductForm = async (req, res) => {
  try {
    const { id } = req.params;
    const product = await Product.findById(id);
    res.render("products-form", { product });
  } catch (error) {
    res.render("error", { error: error.message });
  }
};
```

Update — updateProduct()

Second half of an edit: receives the submitted form and updates the matching document with findByIdAndUpdate(). The result is not used, so { new: true } is not needed. The browser is then redirected to the list.

Syntax

```
Product.findByIdAndUpdate(<id>, <update>)
```

Example

```
const updateProduct = async (req, res) => {
  try {
    const { id } = req.params;
    const { name, description, category, price } = req.body;
    await Product.findByIdAndUpdate(id, {
      name,
      description,
      category,
      price,
    });
    res.redirect("/");
  } catch (error) {
    res.render("error", { error: error.message });
  }
};
```

Delete — deleteProduct()

Removes the product with findByIdAndDelete() and redirects to the list.

Syntax

```
Product.findByIdAndDelete(<id>)
```

Example

```
const deleteProduct = async (req, res) => {
  try {
    const { id } = req.params;
    await Product.findByIdAndDelete(id);
    res.redirect("/");
  } catch (error) {
    res.render("error", { error: error.message });
  }
};
```

5. Request Flow

Each write action follows the Post/Redirect/Get pattern: the form is submitted with POST, the controller changes the data, and a redirect sends the browser back to GET / so a page refresh does not resubmit the form.

Adding a Product

```
GET /products-form
   ↓
inline handler  →  render products-form (product: null)
   ↓
```

```
User fills the form and submits
   ↓
POST /add-product
   ↓
express.urlencoded parses the form into req.body
   ↓
createProduct  →  new Product(...).save()
   ↓
res.redirect("/")
   ↓
GET /  →  readProducts  →  render products
```

Editing a Product

```
GET /products-form/:id
   ↓
updateProductForm  →  Product.findById(id)
   ↓
render products-form (product filled in)
   ↓
User edits the fields and submits
   ↓
POST /add-product/:id
   ↓
updateProduct  →  Product.findByIdAndUpdate(id, ...)
   ↓
res.redirect("/")
```

Deleting a Product

```
GET /delete-product/:id
   ↓
deleteProduct  →  Product.findByIdAndDelete(id)
   ↓
res.redirect("/")
```